



Преподаватель:

Коляда

Никита Владимирович

Обратная связь:

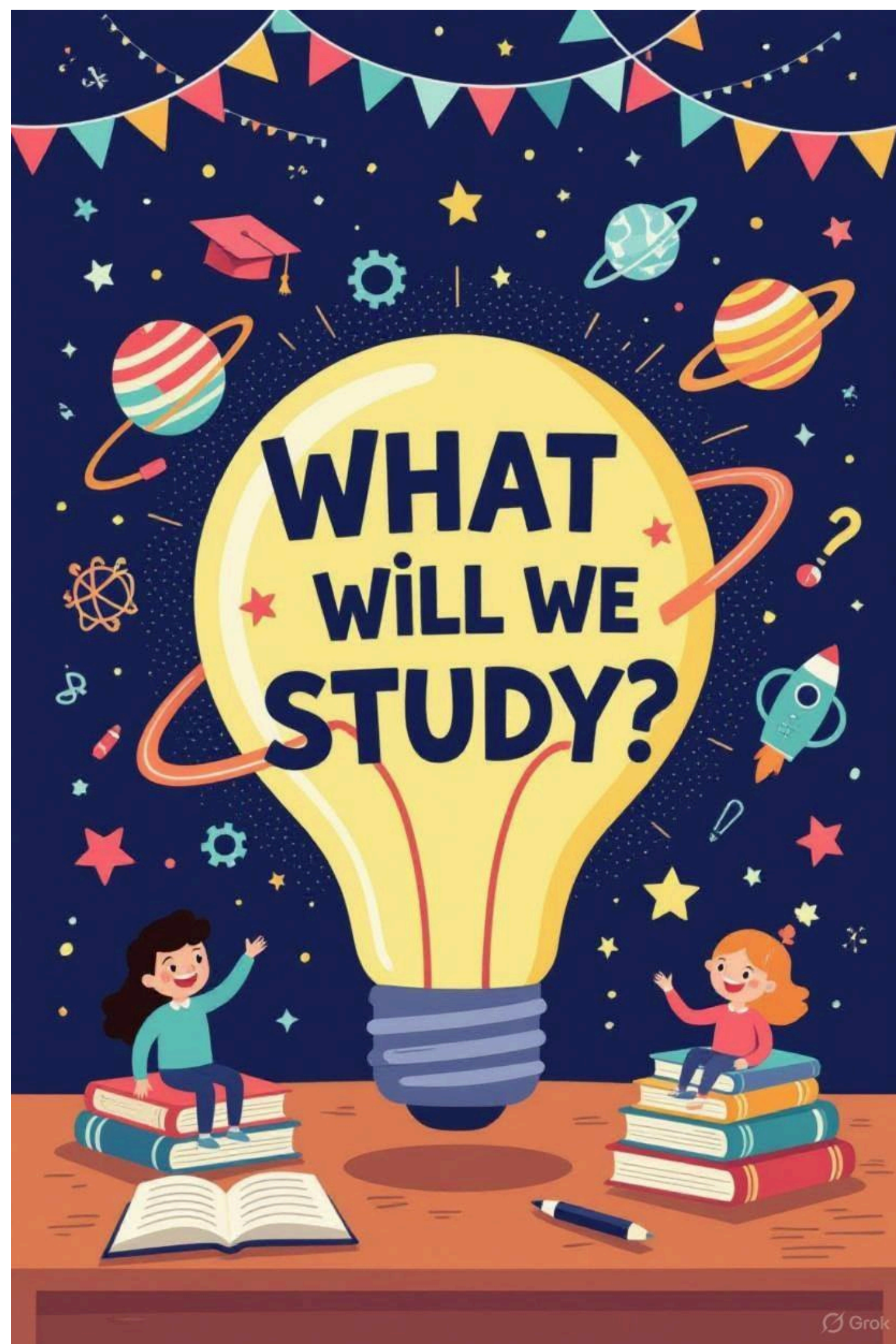
- сообщения на inStudy

ОСНОВЫ ПРОЕКТИРОВАНИЯ АРХИТЕКТУРЫ БАЗЫ ДАННЫХ



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

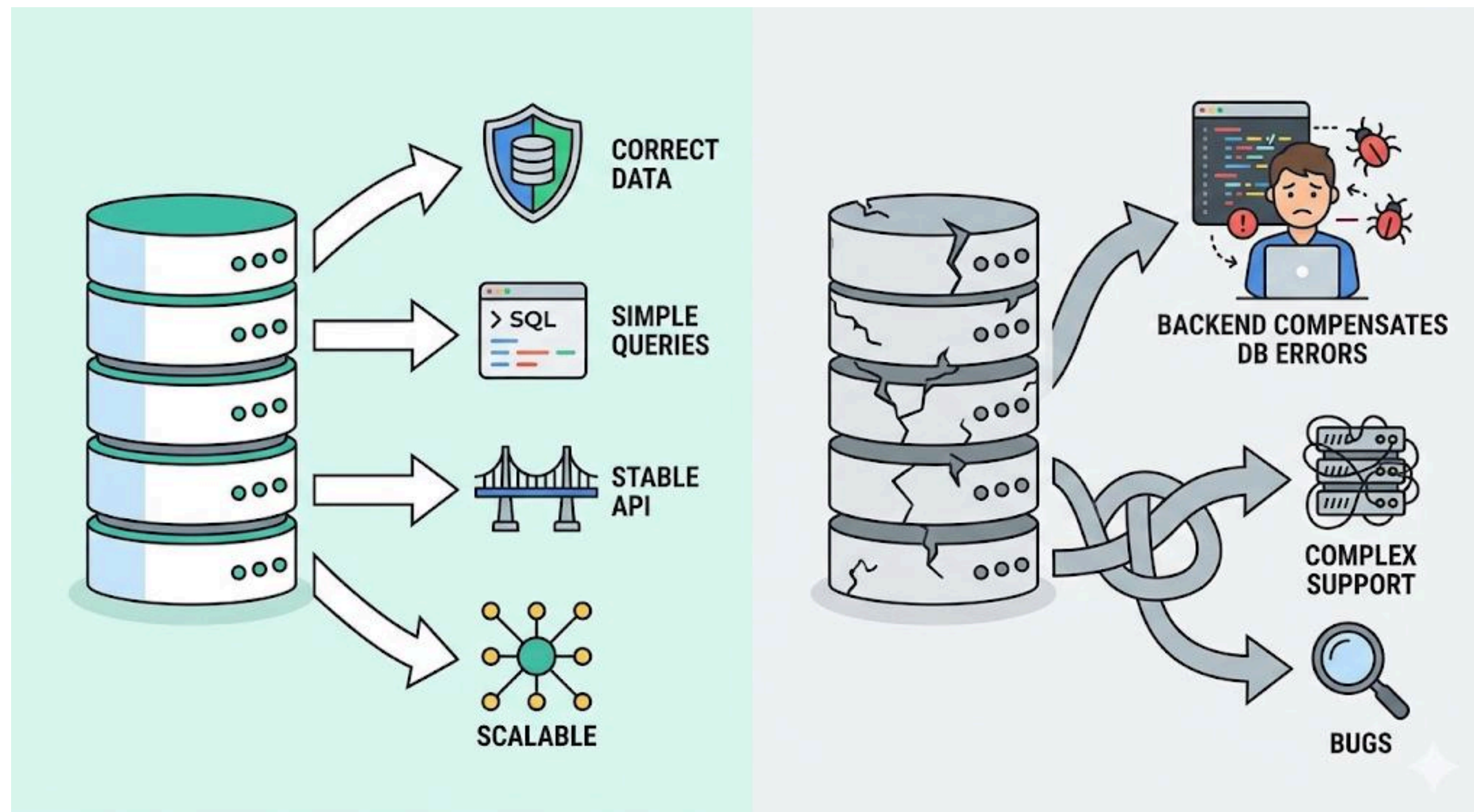


КЛЮЧЕВЫЕ ТЕМЫ

1. Типы данных и их роль в базе данных
2. Ограничения целостности (constraints): NOT NULL, UNIQUE, CHECK, DEFAULT
3. Зачем нужны уникальные идентификаторы
4. Первичный ключ (Primary Key) и требования к нему
5. Натуральные и суррогатные ключи: преимущества и риски
6. Внешний ключ (Foreign Key) и связи между таблицами
7. Целостность ссылок (Referential Integrity)
8. Каскадные операции: ON DELETE / ON UPDATE
9. Типы связей между сущностями: 1:1, 1:N, M:N
10. Промежуточные таблицы (junction tables) для M:N
11. JOIN-запросы и использование связей в SQL
12. Дублирование данных и аномалии обновления
13. Логика нормализации и разделение справочников и транзакционных данных
14. Денормализация: когда допустима

ЗАЧЕМ ПРОЕКТИРОВАТЬ АРХИТЕКТУРУ БД

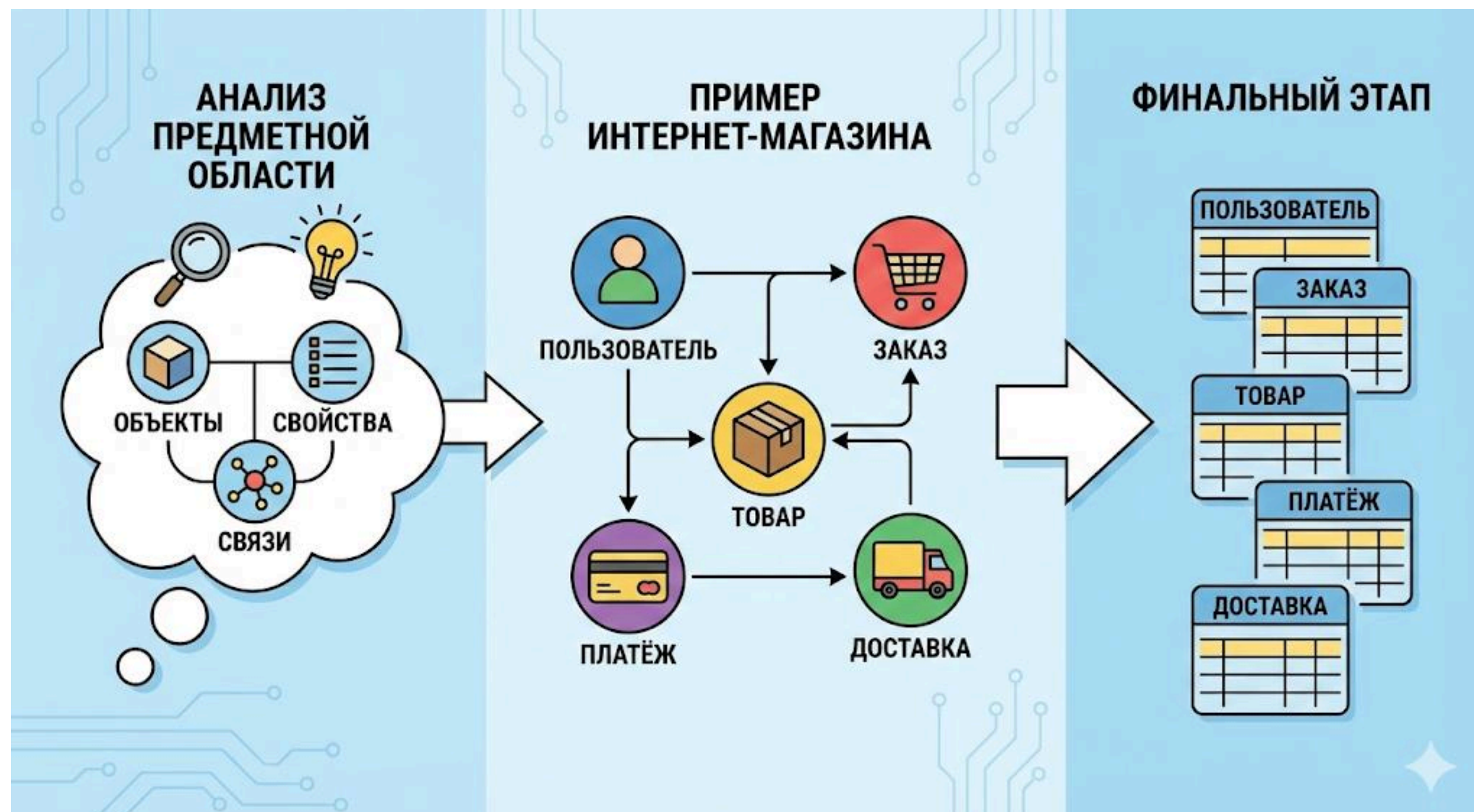
ПОЧЕМУ СТРУКТУРА ДАННЫХ КРИТИЧНА ДЛЯ BACKEND



1. Архитектура базы данных определяет:
 - а. как быстро выполняются запросы
 - б. насколько просто добавлять новые функции
 - в. можно ли избежать дублирования данных
 - г. насколько безопасно и надёжно хранится информация
2. Ошибки на этапе проектирования приводят к сложной поддержке, медленной работе и необходимости полной переработки системы в будущем.
3. Поэтому перед созданием таблиц всегда выполняется проектирование структуры данных.

ОТ БИЗНЕС-ЛОГИКИ К ТАБЛИЦАМ

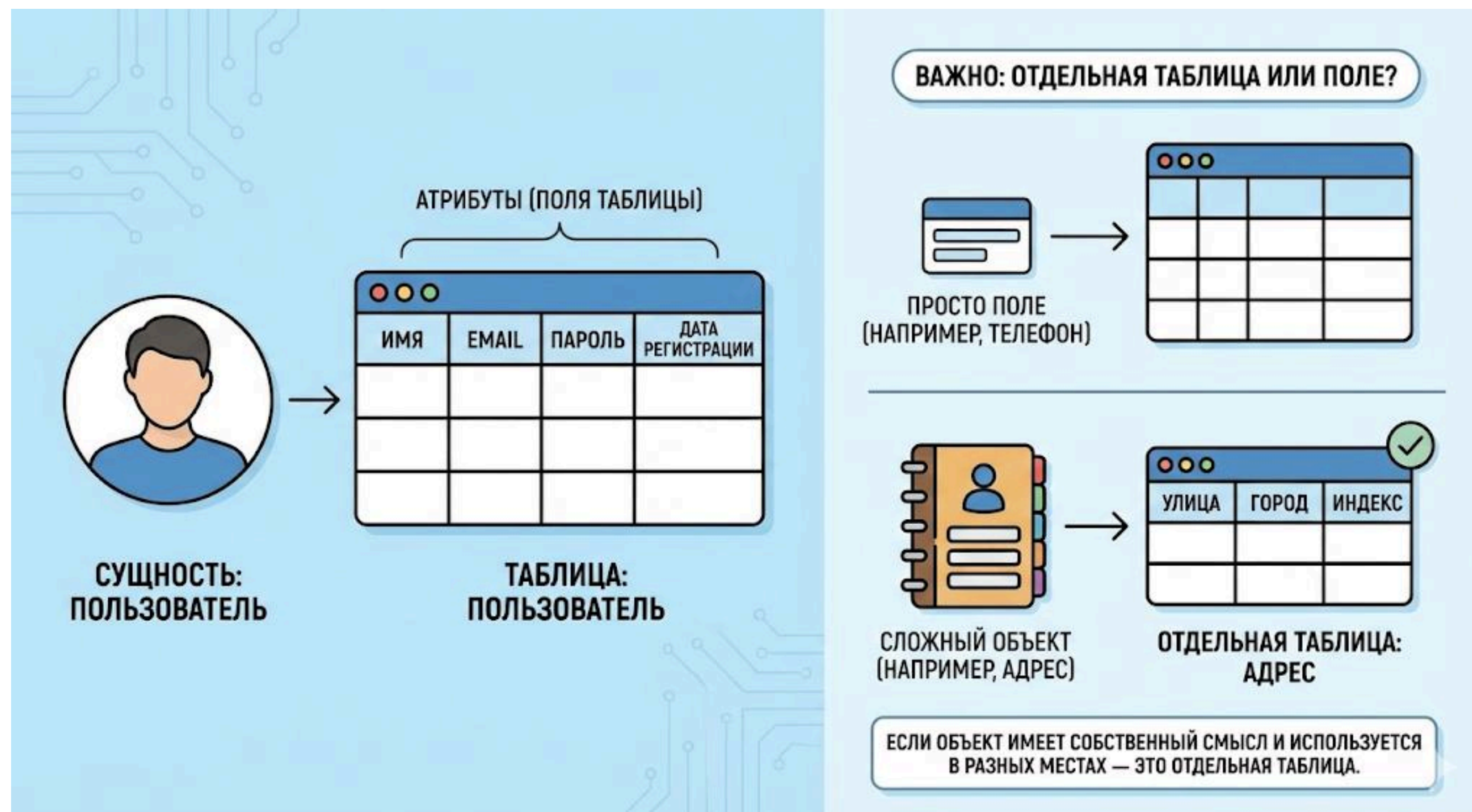
НАЧИНАЕМ С АНАЛИЗА ПРЕДМЕТНОЙ ОБЛАСТИ



1. Проектирование БД начинается не с таблиц, а с понимания:
2. • какие объекты существуют в системе
 - какие у них свойства
 - как они связаны друг с другом
3. Например, в интернет-магазине это могут быть:
4. Пользователь, Заказ, Товар, Платёж, Доставка.
5. Каждый такой объект в дальнейшем становится отдельной таблицей.
6. Этот этап называется анализом предметной области.

СУЩНОСТИ И АТРИБУТЫ

ЧТО СТАНЕТ ТАБЛИЦЕЙ, А ЧТО — ПОЛЕМ



1. Сущность — это объект, который хранится в БД как отдельная таблица.

Атрибут — это свойство сущности, которое становится полем таблицы.

2. Пример:

- Сущность: Пользователь

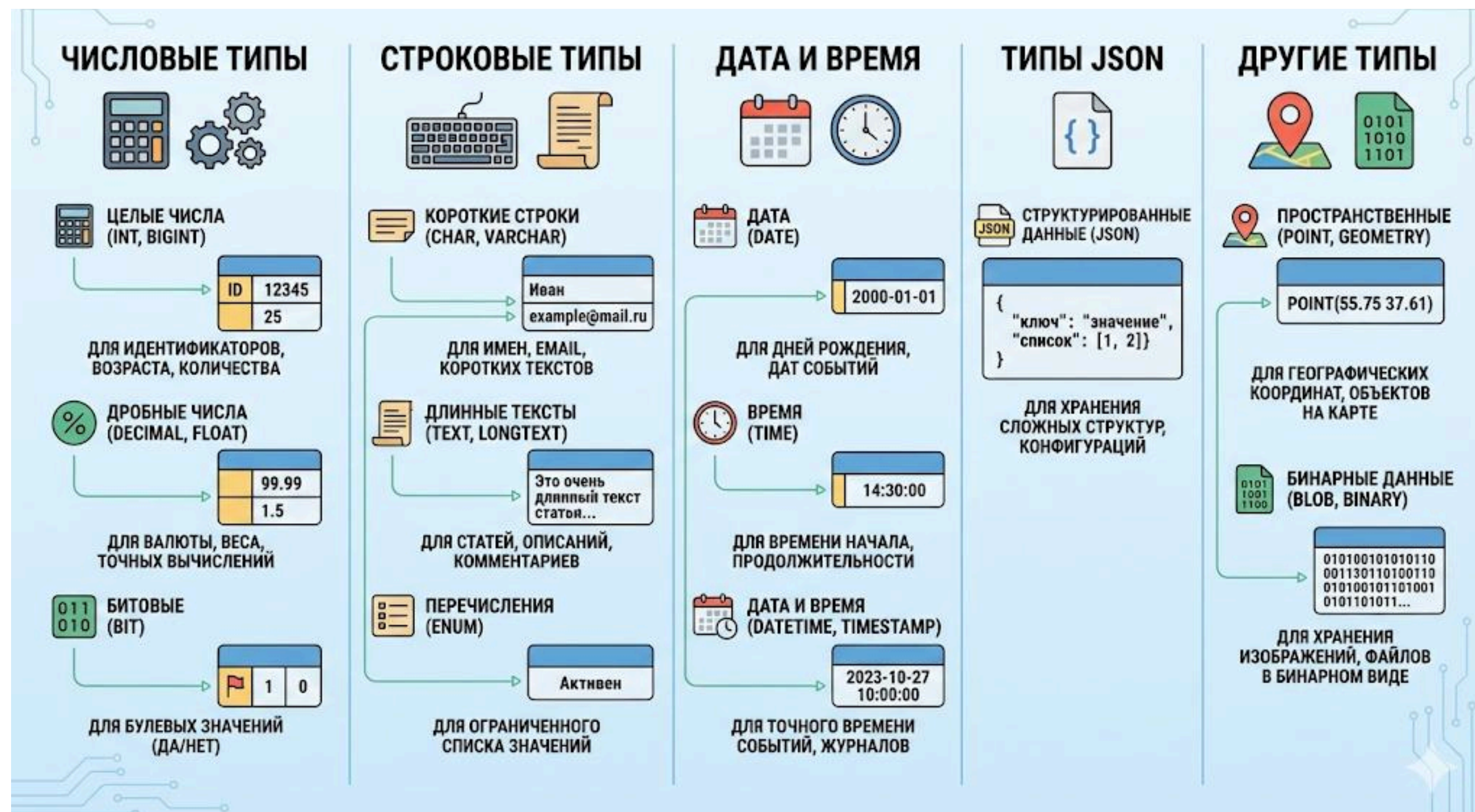
- Атрибуты: имя, email, пароль, дата регистрации

3. Важно:

если объект имеет собственный смысл и используется в разных местах системы — это отдельная таблица, а не просто поле.

ТИПЫ ДАННЫХ

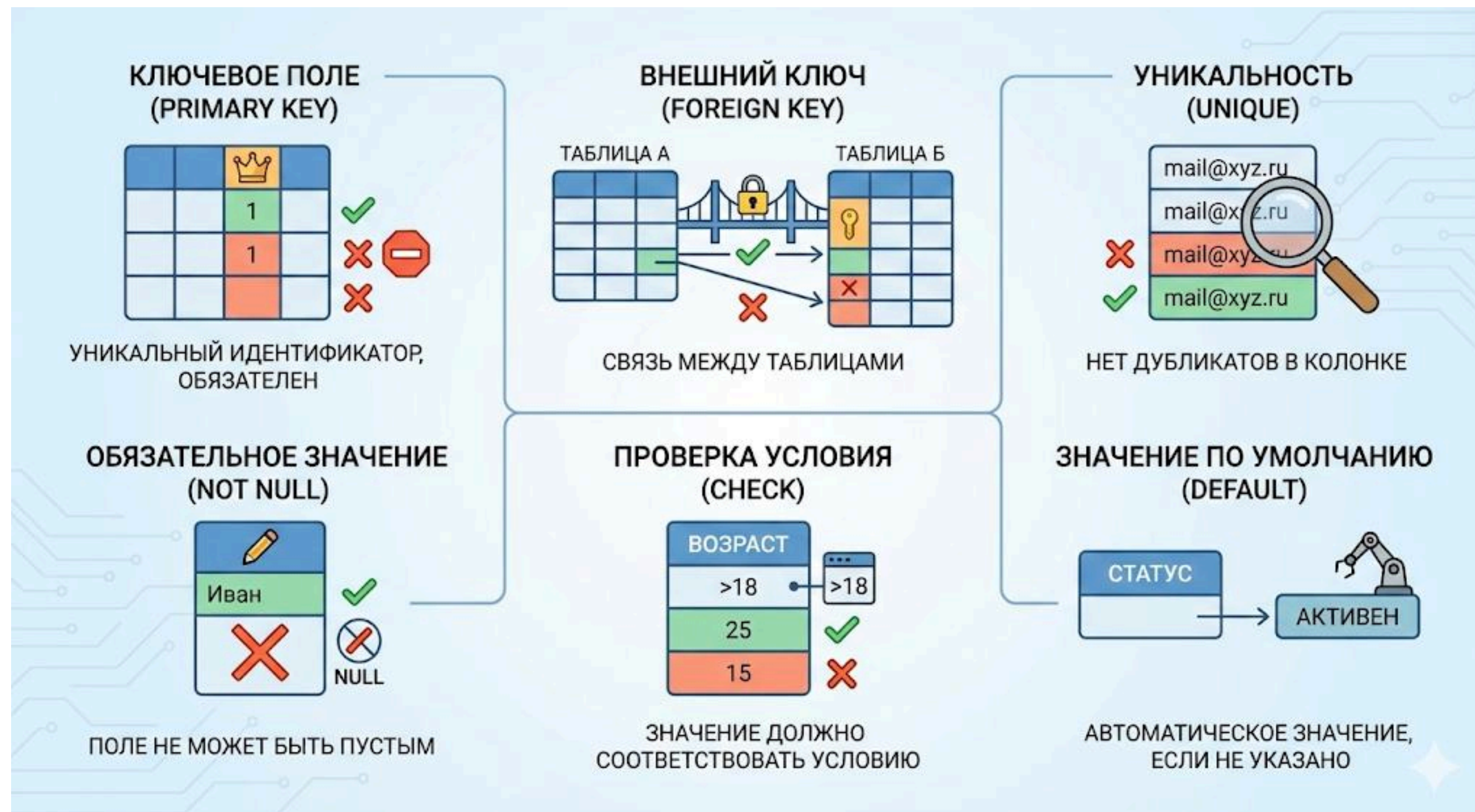
РОЛЬ ТИПОВ ДАННЫХ В БАЗЕ ДАННЫХ



1. Тип данных определяет, какие значения может хранить поле, сколько памяти оно занимает и как быстро выполняются операции над этими данными.
2. Корректный выбор типов данных:
 - предотвращает логические ошибки
 - улучшает производительность
 - облегчает проверку данных
3. Например, цены должны храниться в DECIMAL, а не в INT или FLOAT, даты – в DATE или DATETIME, а не в строках. Типы данных – это первая линия защиты целостности информации.

ОГРАНИЧЕНИЯ ЦЕЛОСТНОСТИ (CONSTRAINTS)

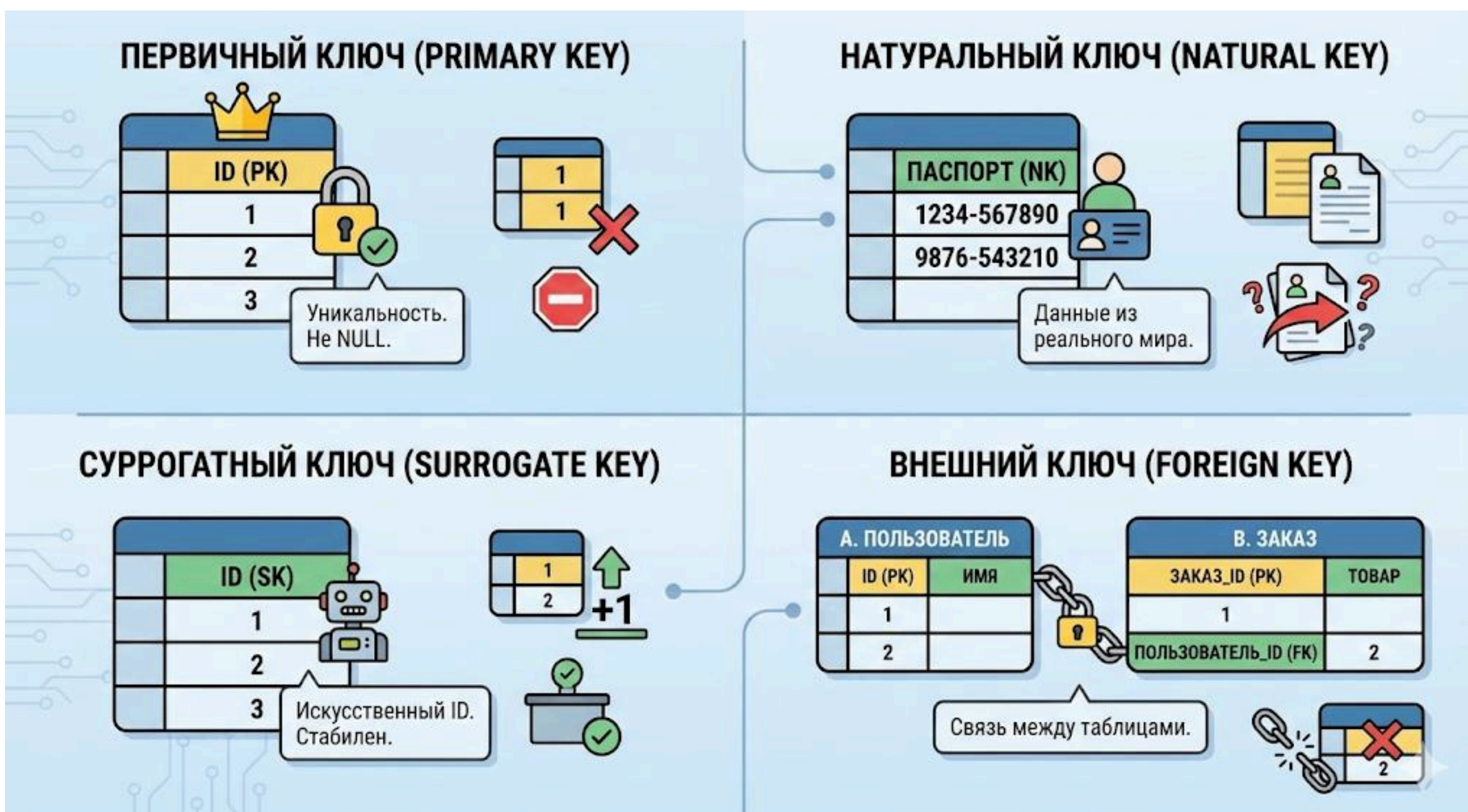
КОНТРОЛЬ КОРРЕКТНОСТИ ДАННЫХ НА УРОВНЕ БД



1. Constraints – это правила, которые база данных применяет при вставке и обновлении данных.
2. Основные ограничения:
 - NOT NULL – значение обязательно
 - UNIQUE – значение не должно повторяться
 - DEFAULT – значение по умолчанию
 - CHECK – проверка условия
3. Ограничения позволяют выявлять ошибки до того, как данные попадут в систему, и не зависят от корректности работы backend-приложения.

ЗАЧЕМ НУЖНЫ УНИКАЛЬНЫЕ ИДЕНТИФИКАТОРЫ

КАК ОДНОЗНАЧНО ОПРЕДЕЛЯТЬ ЗАПИСИ



1. Каждая запись в таблице должна иметь уникальный идентификатор, чтобы:
 - на неё можно было сослаться из других таблиц
 - можно было безопасно обновлять и удалять данные
 - можно было строить связи между сущностями
2. Без идентификаторов невозможно обеспечить корректную работу связей и поддерживать целостность данных.

ЦЕЛОСТНОСТЬ ССЫЛОК (REFERENTIAL INTEGRITY)

ГАРАНТИЯ КОРРЕКТНЫХ СВЯЗЕЙ



1. Referential Integrity означает, что каждая ссылка между таблицами:
 - указывает на реально существующую запись
 - не может быть нарушена случайно
2. Например, заказ не может существовать без пользователя, если между ними задан внешний ключ.
3. Это защищает данные даже при ошибках в backend-коде.

УДАЛЕНИЕ СВЯЗАННЫХ ДАННЫХ

ЧТО ПРОИСХОДИТ ПРИ УДАЛЕНИИ ЗАПИСИ



1. При удалении записи, на которую ссылаются другие таблицы, возможны варианты:

- запретить удаление
- удалить связанные записи
- обнулить ссылки

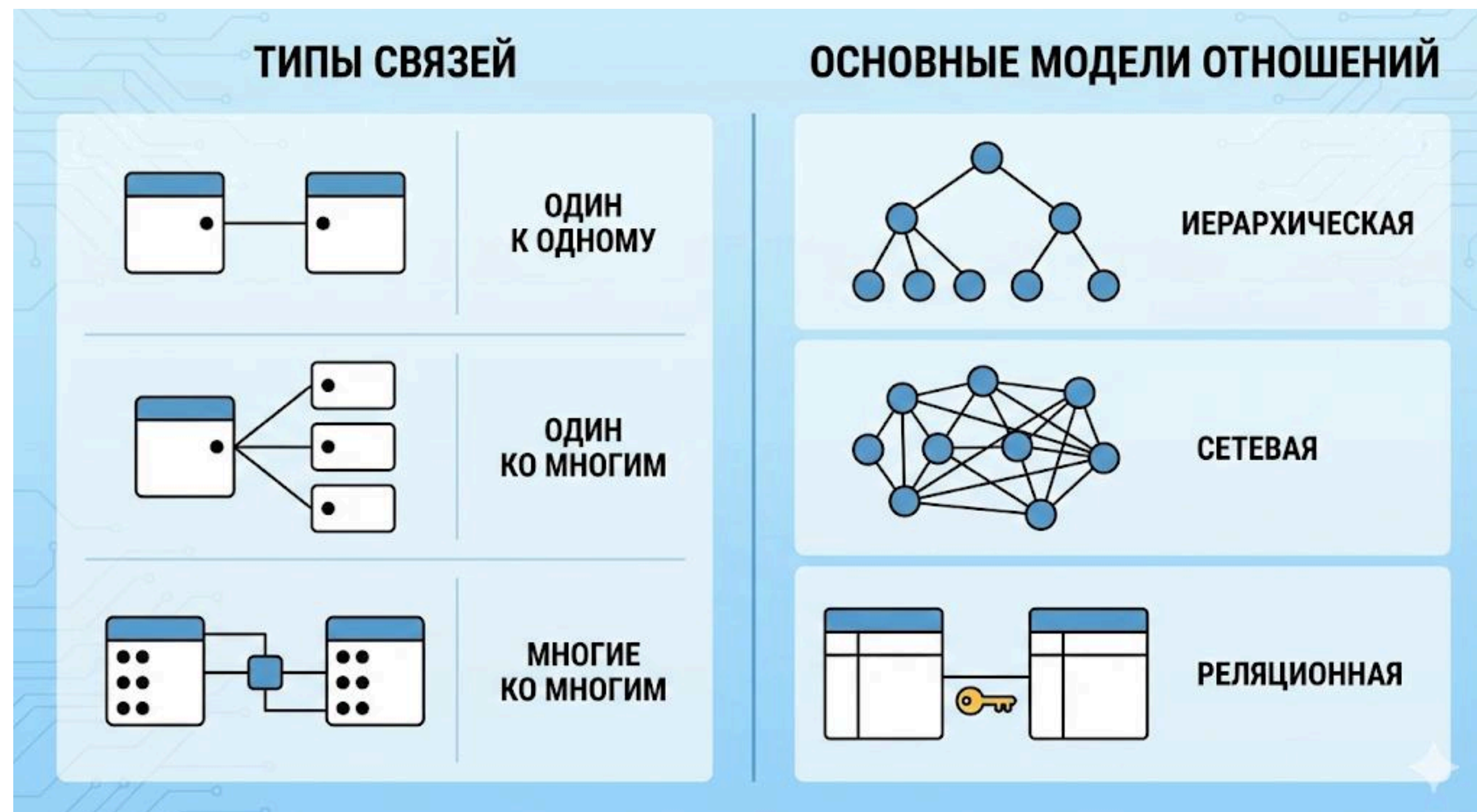
2. Каскадные операции позволяют автоматически изменять связанные данные:

- ON DELETE CASCADE – удалять дочерние записи
- ON DELETE RESTRICT – запрещать удаление
- ON UPDATE CASCADE – обновлять ключи

3. Каскады удобны, но при неправильном применении могут привести к массовому удалению данных. Поэтому их используют только там, где это логически оправдано.

ЛОГИКА СВЯЗЕЙ МЕЖДУ СУЩНОСТЯМИ

ТИПЫ СВЯЗЕЙ



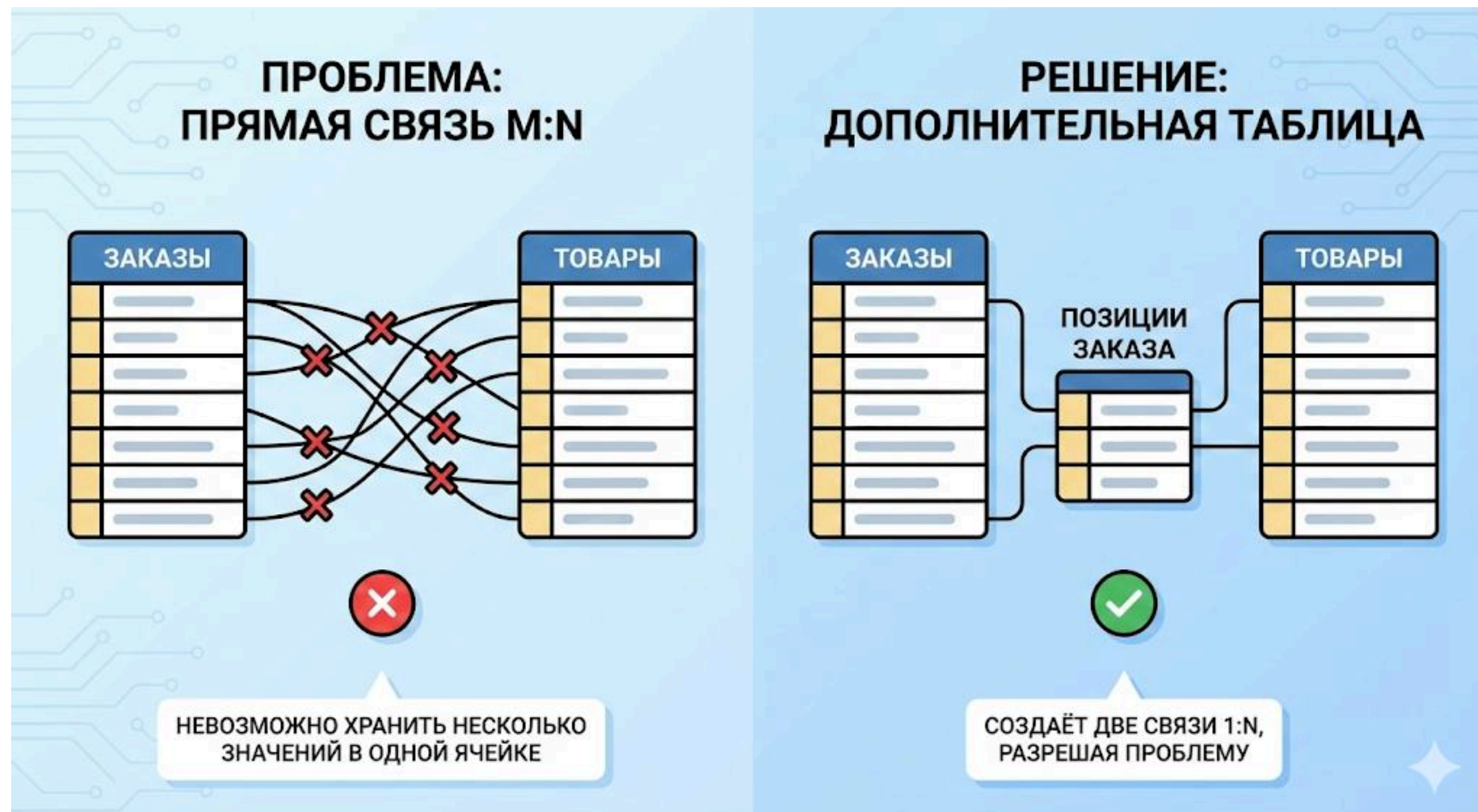
1. Существует три базовых типа связей:

- 1:1 — один к одному
- 1:N — один ко многим
- M:N — многие ко многим

2. Тип связи определяет структуру таблиц и способ хранения данных.

КОГДА НЕЛЬЗЯ ИСПОЛЬЗОВАТЬ M:N НАПРЯМУЮ

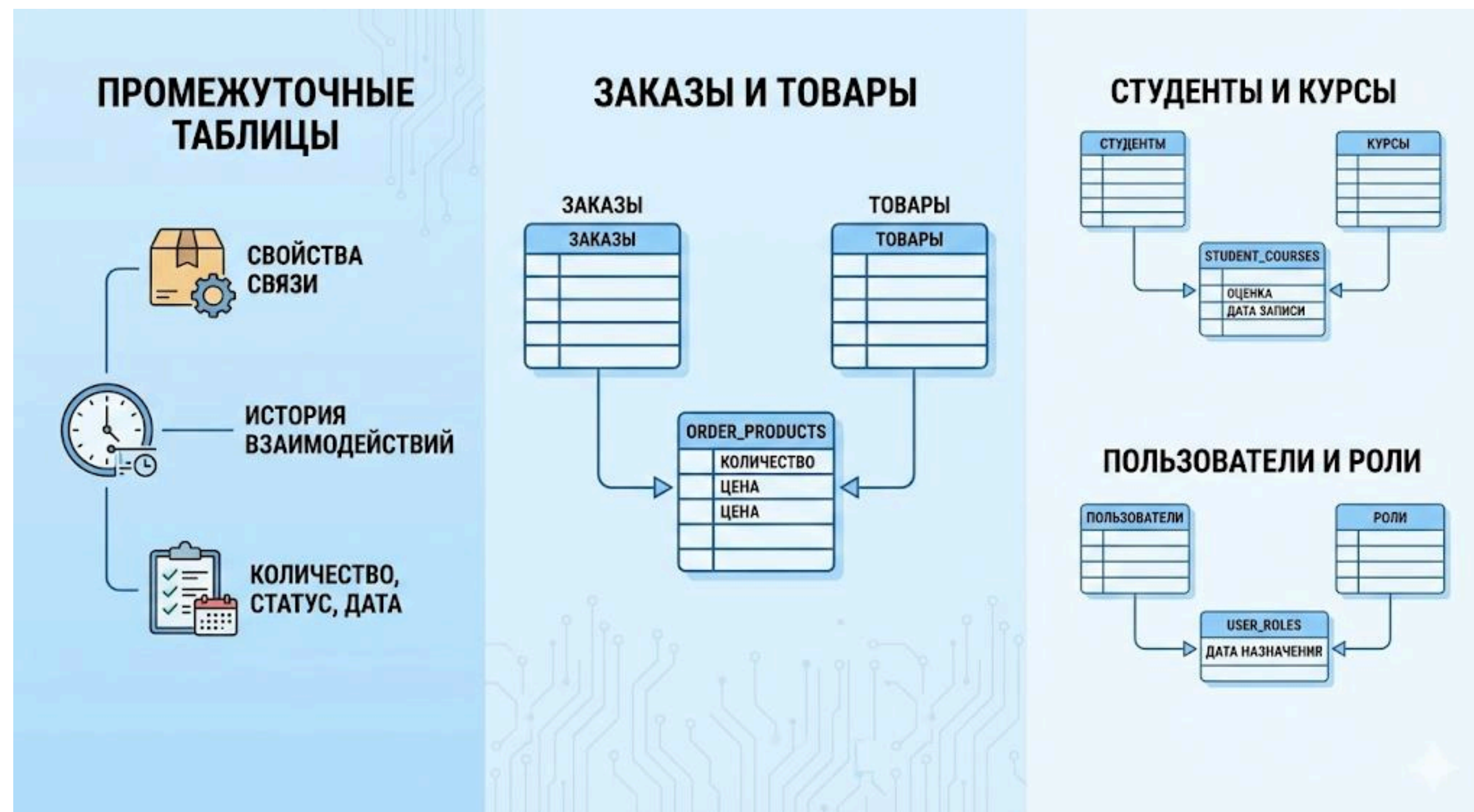
ПОЧЕМУ НУЖНА ДОПОЛНИТЕЛЬНАЯ ТАБЛИЦА



1. Реляционные БД не поддерживают прямые связи многие-ко-многим.
2. Для их реализации создаётся промежуточная таблица, которая:
 - содержит внешние ключи на обе сущности
 - может хранить дополнительные атрибуты связи
3. Без неё невозможно корректно представить такую модель данных.

ПРОМЕЖУТОЧНЫЕ ТАБЛИЦЫ

JUNCTION TABLES КАК ЧАСТЬ АРХИТЕКТУРЫ



1. Промежуточные таблицы используются, когда:

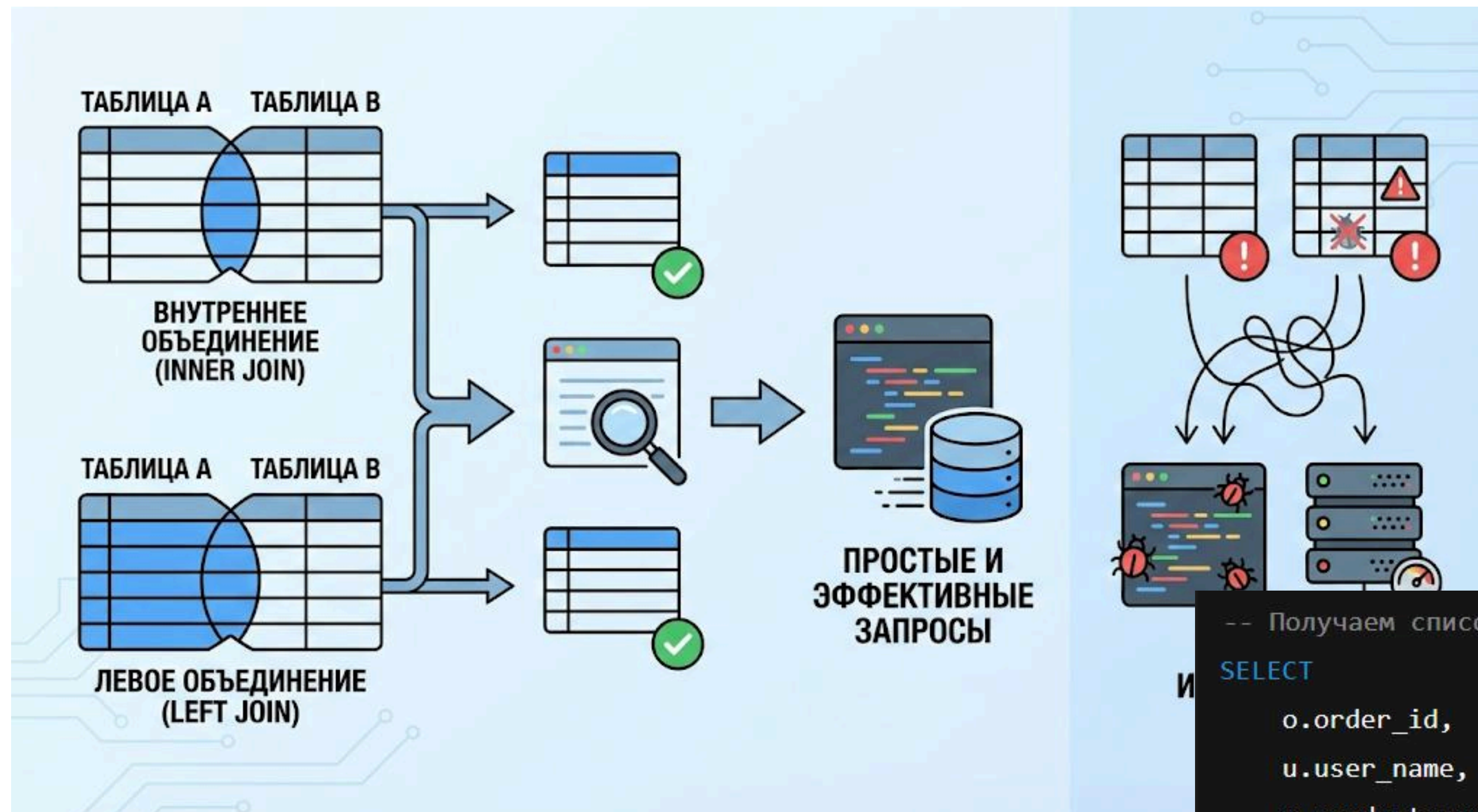
- связь имеет собственные свойства
- требуется хранить историю взаимодействий
- необходимо учитывать количество, статус, дату

2. Примеры:

- Промежуточная таблица `order_products` связывает заказы и товары в системе.
- Таблица `student_courses` соединяет студентов и курсы.
- Таблица `user_roles` связывает пользователей и роли в системе.

РОЛЬ СВЯЗЕЙ В JOIN-ЗАПРОСАХ

КАК ДАННЫЕ ОБЪЕДИНЯЮТСЯ ПРИ ВЫБОРКЕ

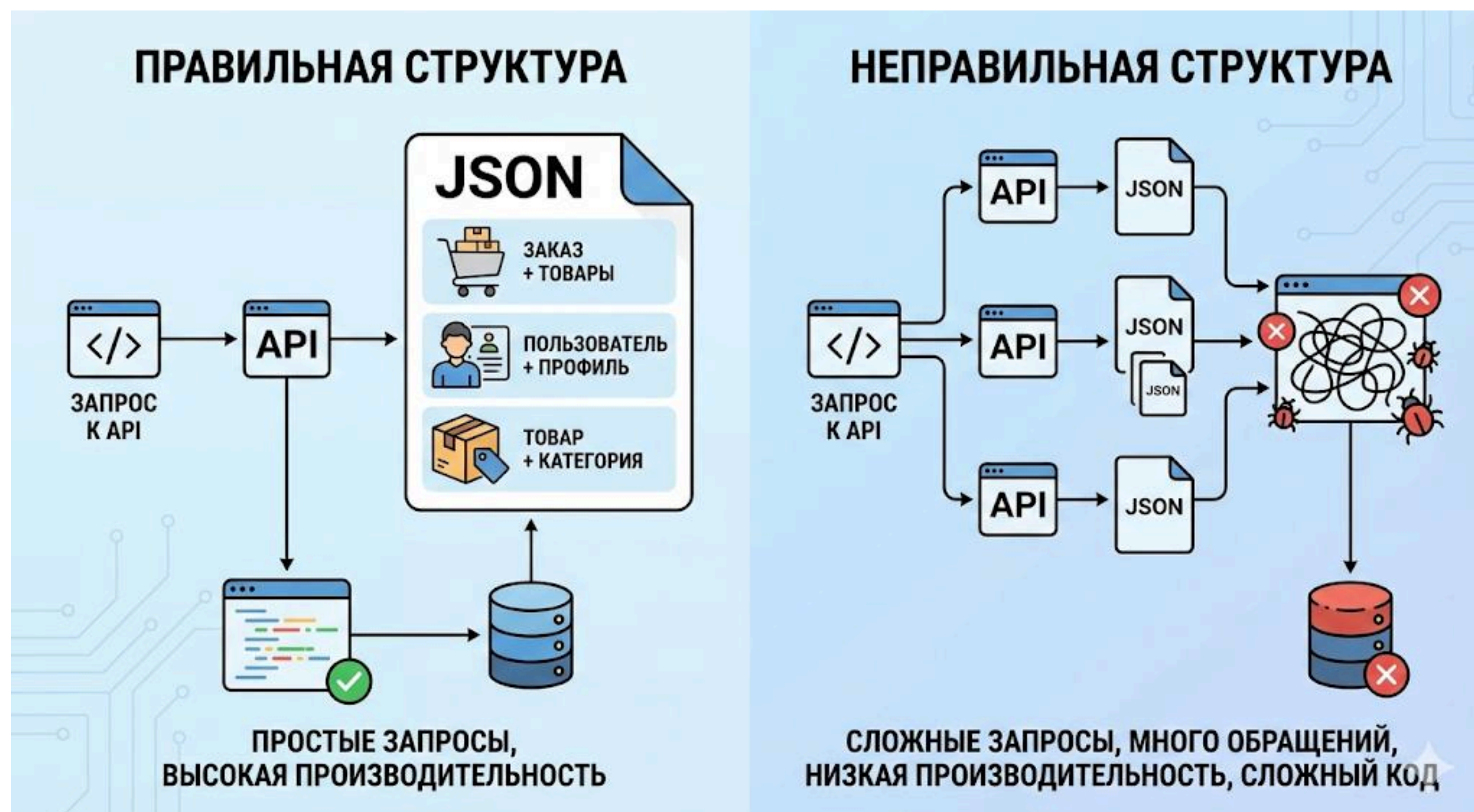


1. Связи позволяют получать данные из нескольких таблиц с помощью JOIN:
 - INNER JOIN — только связанные записи
 - LEFT JOIN — все записи основной таблицы
2. Корректные связи позволяют строить простые и эффективные запросы, а их отсутствие приводит к сложной и ошибочной логике в backend.

```
-- Получаем список всех заказов с информацией о пользователе и товарах
SELECT
  o.order_id,
  u.user_name,
  p.product_name,
  op.item_quantity
FROM orders o
JOIN users u ON o.user_id = u.user_id -- связываем заказ с пользователем
JOIN order_products op ON o.order_id = op.order_id -- связываем заказ с промежуточной таблицей
JOIN products p ON op.product_id = p.product_id; -- связываем промежуточную таблицу с товарами
```


ВЛИЯНИЕ СВЯЗЕЙ НА API

КАК СТРУКТУРА БД ВЛИЯЕТ НА BACKEND



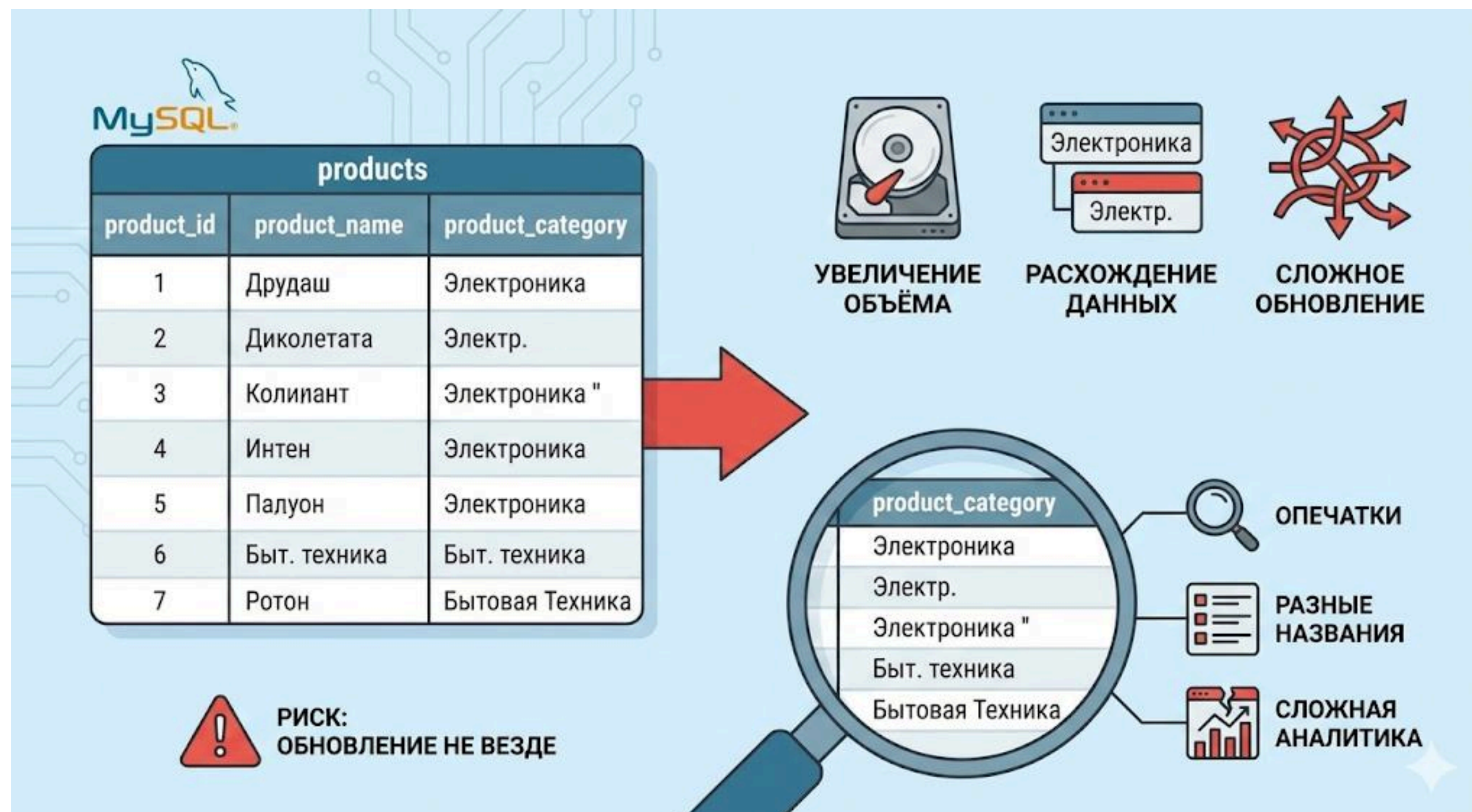
1. API обычно возвращает объединённые данные:

- заказ + товары
- пользователь + профиль
- товар + категория

2. Если связи спроектированы неправильно, backend вынужден выполнять сложные запросы или делать несколько обращений к БД, что ухудшает производительность и усложняет код.

ДУБЛИРОВАНИЕ ДАННЫХ

ПОЧЕМУ ПОВТОРЕНИЯ ОПАСНЫ



1. Дублирование приводит к:

- увеличению объёма БД
- расхождению данных
- сложному обновлению информации

2. Если одно и то же значение хранится в нескольких местах, возникает риск, что обновление будет выполнено не везде.

3. Если категория товара хранится строкой в products: **product_category VARCHAR(45)**

4. мы получаем:

- опечатки
- разные названия
- сложную аналитику

ЧТО ТАКОЕ НОРМАЛИЗАЦИЯ И ЗАЧЕМ ОНА НУЖНА

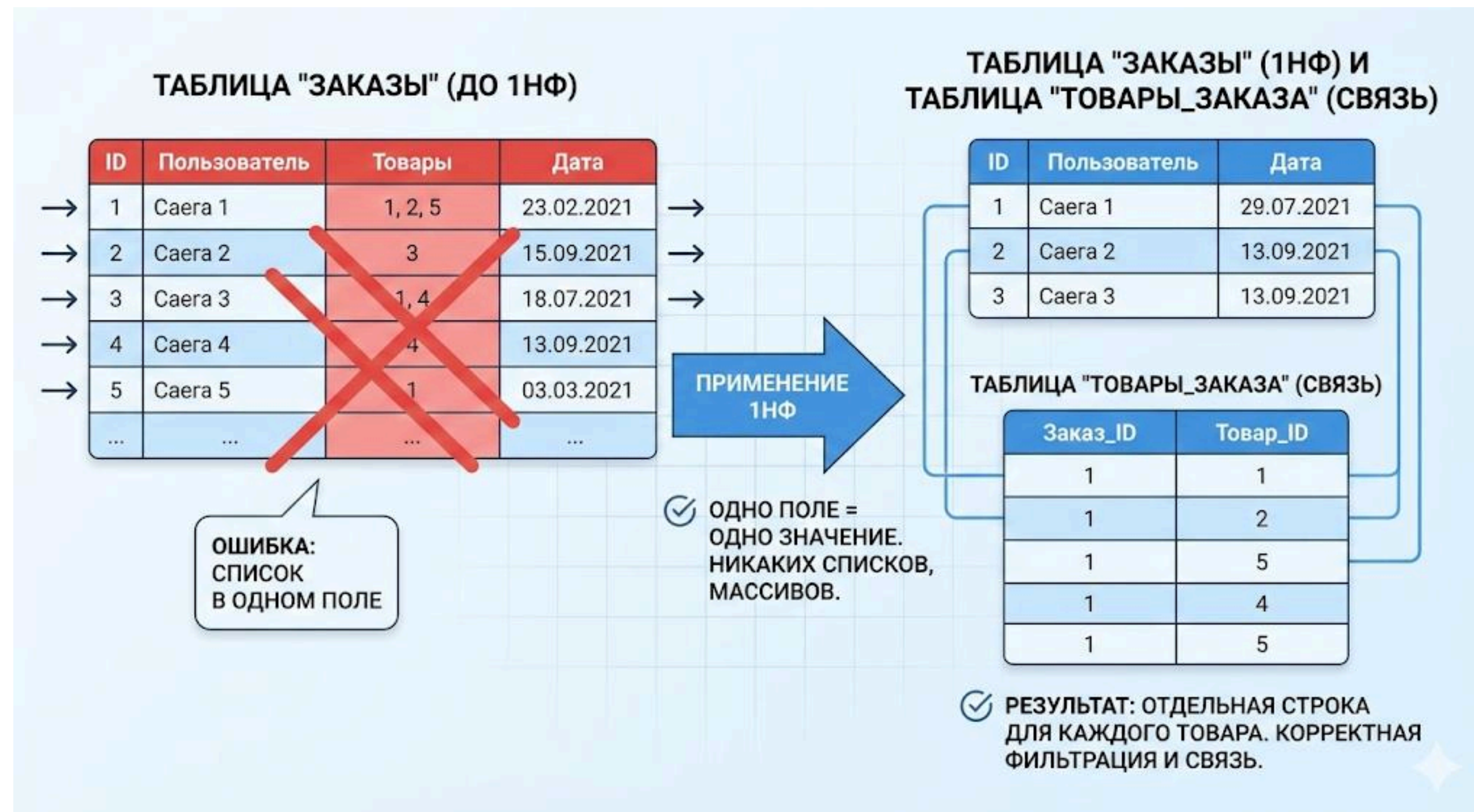
ЗАЧЕМ НОРМАЛИЗОВАТЬ БАЗУ ДАННЫХ



1. Нормализация — это процесс организации таблиц так, чтобы каждая таблица отвечала только за **одну логическую сущность** и не содержала лишних повторений данных.
2. **Цель нормализации** — уменьшить дублирование, упростить обновление данных и снизить риск ошибок.
3. Если одно и то же значение хранится в нескольких местах, при изменении его легко забыть обновить везде, что приводит к рассинхронизации данных.
4. Нормализованная структура делает базу данных более предсказуемой и удобной для развития проекта.

ПЕРВАЯ НОРМАЛЬНАЯ ФОРМА (1NF)

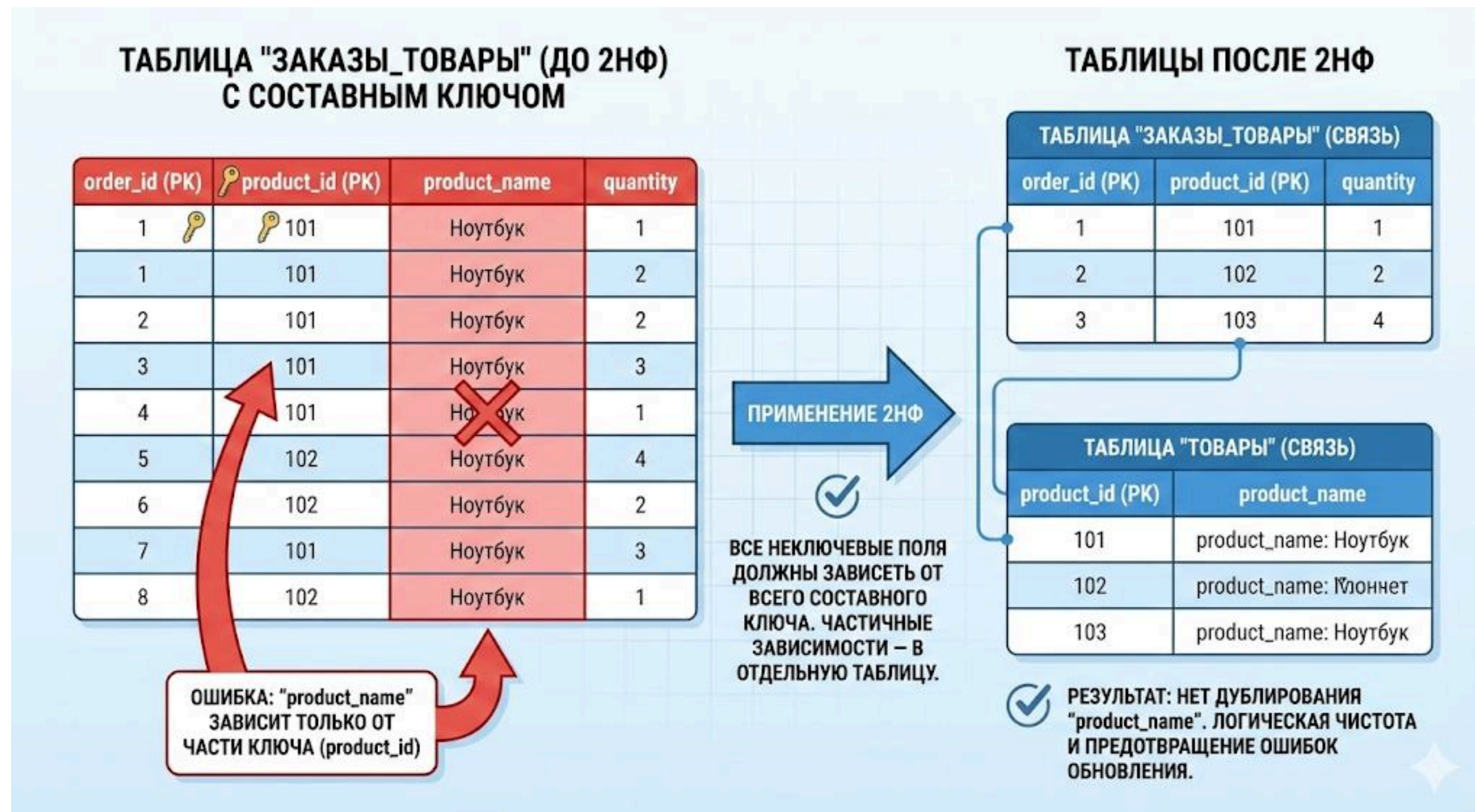
АТОМАРНЫЕ ДАННЫЕ



1. Первая нормальная форма требует, чтобы в каждом поле хранилось **только одно значение**.
2. В таблице **не должно быть списков, массивов и повторяющихся групп столбцов**.
3. Каждая строка должна описывать **один объект**, а каждое поле — **одно его свойство**.
4. Если в одном поле хранится несколько значений, такие данные невозможно корректно фильтровать и связывать с другими таблицами.
5. Практический пример:
Вместо поля **products = "1,2,5"** используется отдельная таблица **order_products**, где **каждая строка — один товар** в заказе.

ВТОРАЯ НОРМАЛЬНАЯ ФОРМА (2NF)

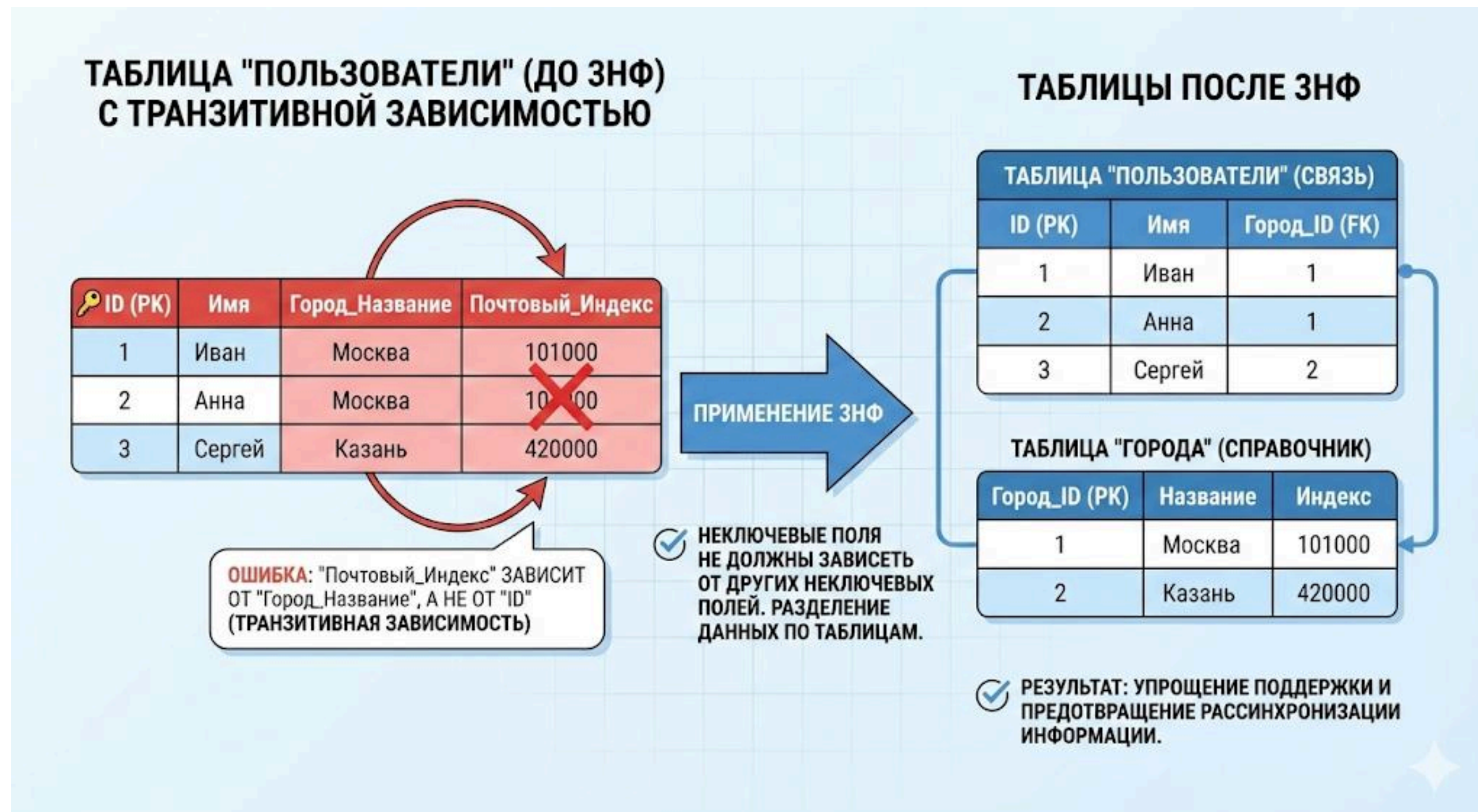
УСТРАНЕНИЕ ЧАСТИЧНЫХ ЗАВИСИМОСТЕЙ



1. Вторая нормальная форма применяется к таблицам с **составным первичным ключом**.
2. Все неключевые поля должны зависеть от всего ключа, а не от его части.
3. Если данные зависят только от одного поля составного ключа, они должны быть вынесены в отдельную таблицу.
4. Это предотвращает дублирование и логические ошибки при обновлении данных.
5. Практический пример:
В таблице **order_products(order_id, product_id)** нельзя хранить **product_name**, так как он зависит только от **product_id** и должен храниться в таблице **products**.

ТРЕТЬЯ НОРМАЛЬНАЯ ФОРМА (3NF)

УСТРАНЕНИЕ ЦЕПОЧЕК ЗАВИСИМОСТЕЙ



1. В третьей нормальной форме неключевые поля не должны зависеть от других неключевых полей.
2. Если одно поле вычисляется или логически следует из другого, такие данные нужно разделять по разным таблицам.
3. Это упрощает поддержку данных и предотвращает рассинхронизацию информации.
4. Каждая таблица должна отвечать только за одну сущность и её собственные свойства.
5. Практический пример:
Если в таблице **пользователей** хранить **city_name** и **city_zip**, то лучше вынести города в таблицу **cities**, а в **users** оставить только **city_id**.

СПРАВОЧНИКИ И ТРАНЗАКЦИИ

РАЗНЫЕ ТИПЫ СУЩНОСТЕЙ – РАЗНЫЕ ТАБЛИЦЫ



1. Справочники:

- категории
- статусы
- роли

2. Транзакционные данные:

- заказы
- платежи
- действия пользователей

3. Их нельзя смешивать, так как они имеют разный жизненный цикл и разные требования к хранению.



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?